SDLC Using Agentic AI

Building an Adaptive AI Software Engineering Organization

Presented By:

Abhishek Raj (M.Tech Intern)

Guide:

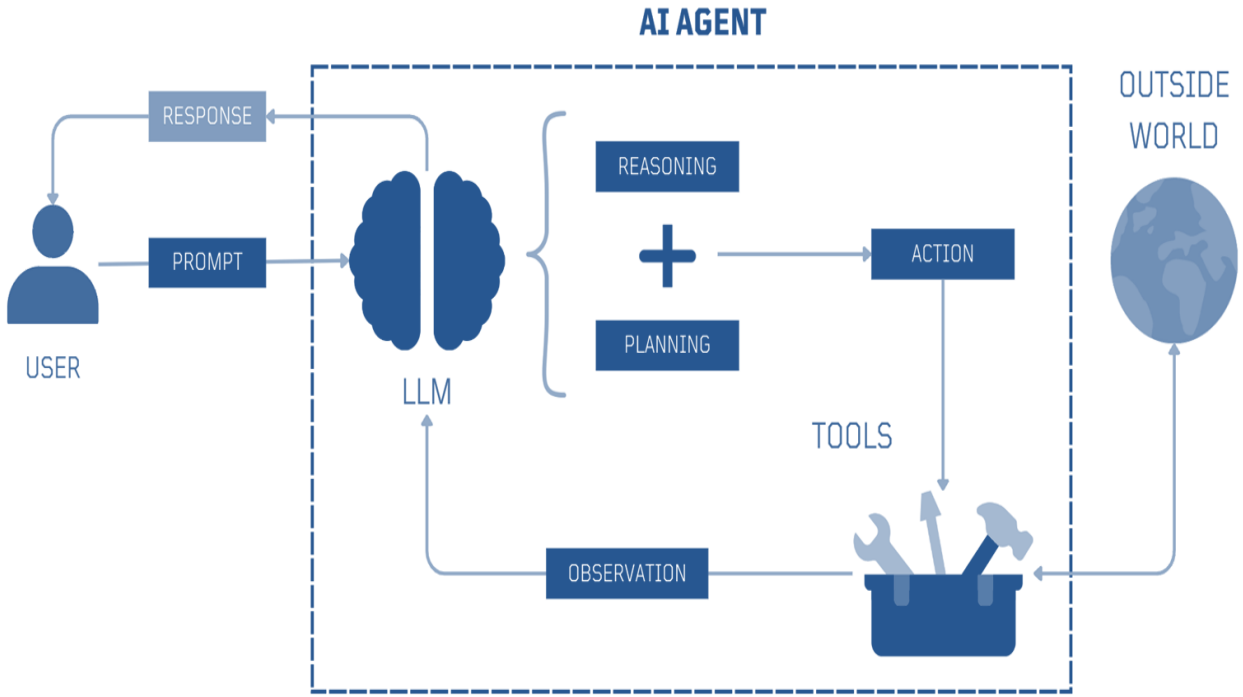
M.B. Sai Charan (Scientist C)

What is an AI Agent?

An AI agent is a system where an LLM operates in a loop: it receives observations from an environment (user messages, tool outputs, sensor data), reasons about what to do next, takes actions (tool calls, code execution, API requests), and iterates until a goal is achieved or it explicitly asks for human input.

An AI agent should have the following characteristics:

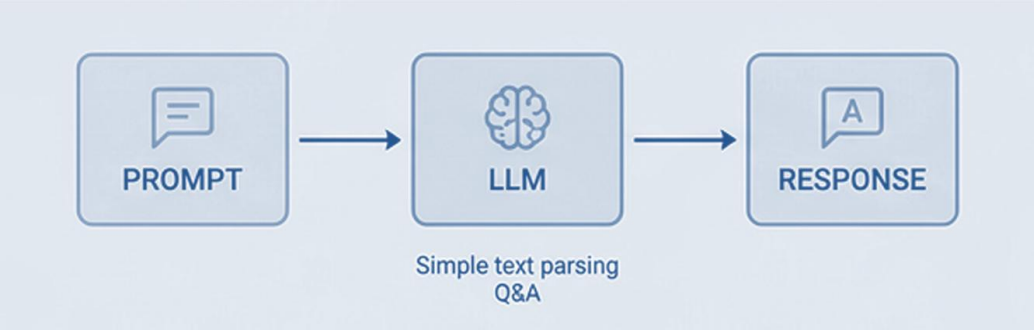
- **Persistence:** An agent must remember what it has done, what failed, and what context was established—across turns, sessions, and even days.
- **Grounding:** The agent must access up-to-date, domain-specific knowledge that was not present in its training data.
- **Action:** The agent must interact with external systems—databases, APIs, file systems, browsers—through well-defined interfaces.
- **Coordination:** Complex tasks often exceed what a single agent can handle; multiple specialized agents must collaborate, delegate, and negotiate.
- **Safety:** Autonomous action requires guardrails, human oversight, and graceful degradation when the agent is uncertain. [6]



Types of AI Agent

1. Reactive / Single-Turn Agents

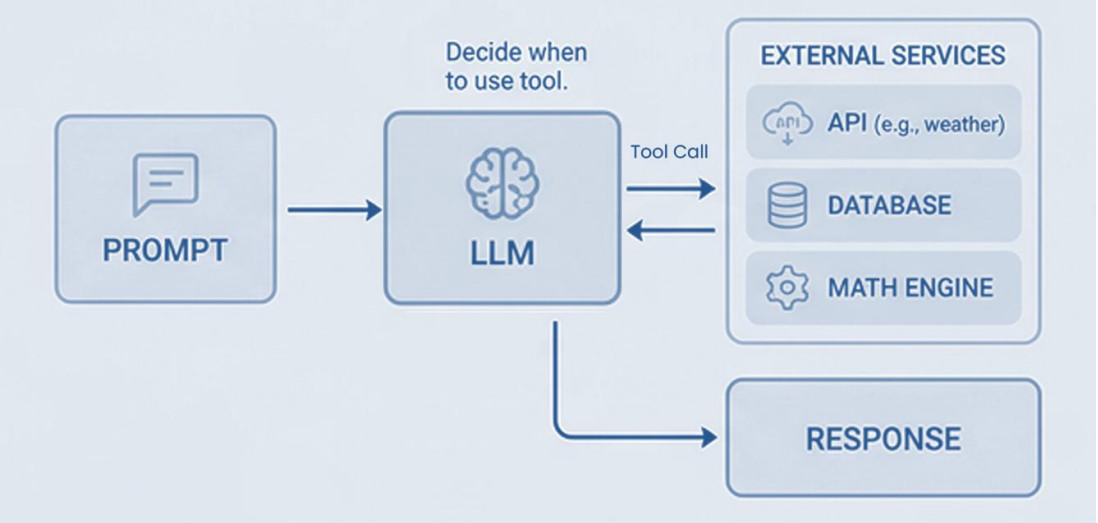
These ai agents respond strictly to one prompt at a time without multi-step foresight. They rely on simple text parsing, direct Q&A, and quick context matching. These agents do not maintain implicit state across sessions or perform long-term planning, making them highly efficient for immediate, single-step resolutions.



Example Use Cases: Basic customer support chatbots, simple text summarization.

2. Tool-Calling Agents

These ai agents use the LLM to decide when and how to call external services. They can call APIs (e.g., weather, stocks), query databases, or execute math. These agents bridge the gap between parametric knowledge and real-time external data execution.



Example Use Cases: Real-time data retrieval assistants, Booking & Scheduling Assistants, etc

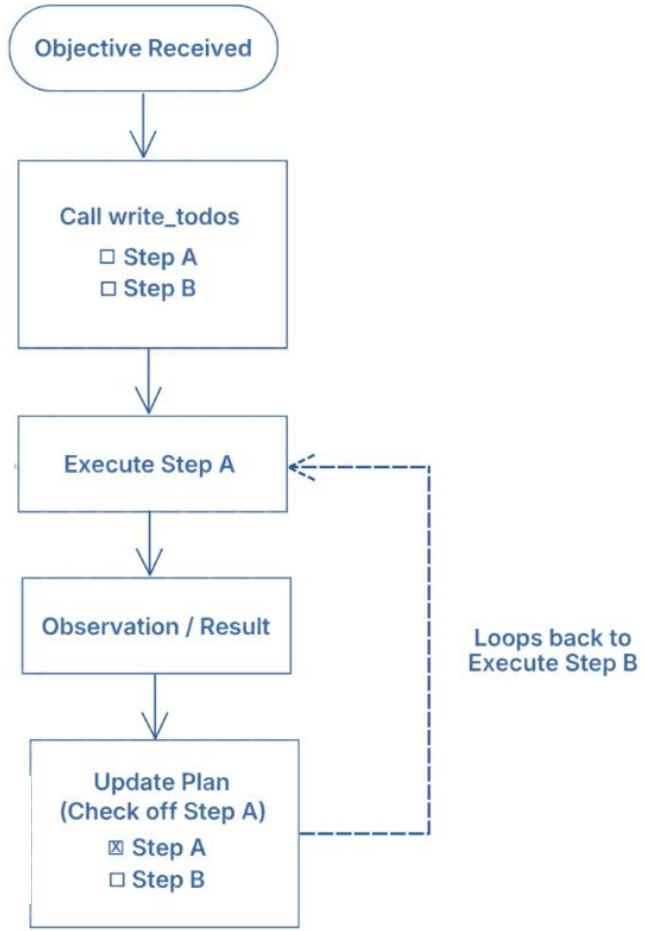
Types of AI Agent (Cont.)

3. Goal-Oriented / Autonomous Agents

Given a high-level goal, they independently break it down into a multi-step plan. Goal-based ai agents have an internal model of the world and also a goal or set of goals. These agents search for action sequences that reach their goal and plan these actions before acting on them.

This search and planning improve their effectiveness when compared to other simple ai agents.

They maintain persistent memory and adapt their strategies based on intermediate outcomes to ensure successful goal completion.



> **Example Use Cases:** Comprehensive market research, autonomous end-to-end software testing, etc

Architectures

Workflow vs Autonomous Agent Design Patterns

Workflow Patterns

Execution paths, step ordering, and condition gates are explicitly defined in code. LLM calls serve as reliable processing nodes within the system pipeline. [6]

- Higher predictability and testability
- Lower latency and controlled token expenditure
- Ideal for standardized, step-by-step transformations

> **Primary Patterns:** Prompt Chaining, Sequential Workflows, Routing, Parallelization, Orchestrator-Workers, Evaluator-Optimizer.

Autonomous Agent Pattern

The agent autonomously determines its execution path based on environment feedback, goal planning, reflection, and tool execution. [6]

- High adaptability to open-ended goals
- Dynamic tool selection and runtime recovery
- Handles complex multi-step reasoning environments

> **Primary Patterns:** ReAct (Reason+Act), Planning Agents, Reflection / Self-Critique, Multi-Agent Collaboration.

Workflow Patterns

1. Prompt Chaining

Decomposes a task into a fixed sequence of LLM calls where the output of one step becomes the input context for the next, with validation gates, ensuring higher precision and intermediate quality control at each stage. [6]

When to use: Tasks that are naturally sequential—content generation, data transformation, multi-stage analysis.

> **Use Cases:** Multi-stage reasoning, document analysis, data transformation.



Fig: Prompt chaining with quality gates. Each step is a separate LLM call. Gates can be LLM-based or programmatic.[6]

2. Routing

Uses an initial router or classifier to direct incoming requests to specialized agents or processing paths with tailored prompts and tools , optimizing latency and cost by serving simple requests quickly and reserving larger, specialized models for complex tasks. [6]

When to use: Distinct task types with different optimal prompts, tools, or models. Customer support triage, multi-modal input handling.

> **Use Cases:** Customer support triage, query classification

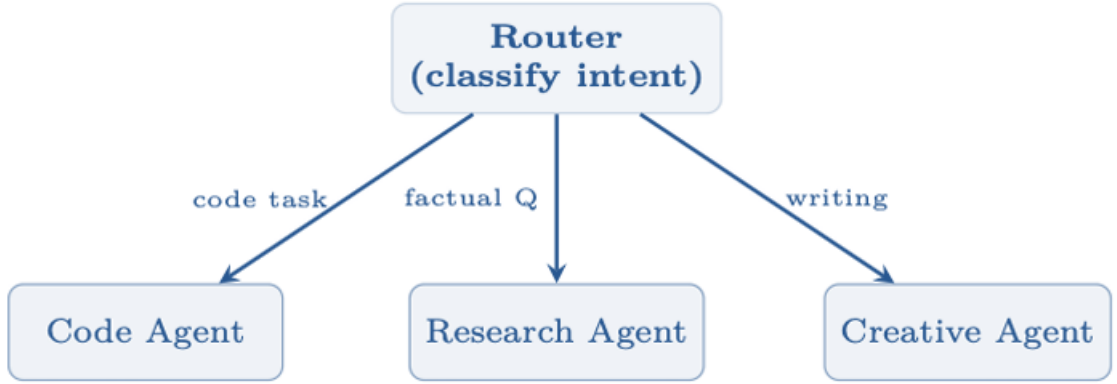


Fig: Routing pattern: input is classified once, then handled by a specialist. [6]

Workflow Patterns (Cont.)

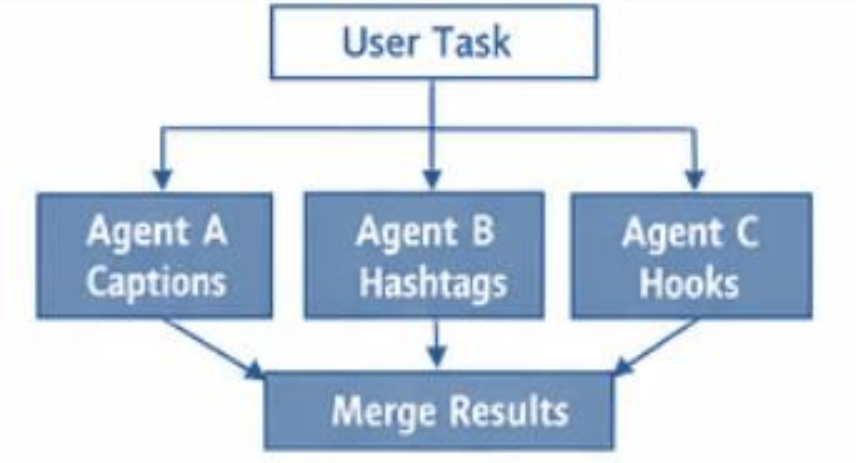
4. Parallelization

Distributes independent subtasks across concurrent LLM calls. Executes simultaneously and aggregates outputs to reduce total system latency , making it ideal for independent evaluation tasks, majority-voting consensus mechanisms, and multi-perspective verification. [6]

It has two sub-patterns:

Sectioning (fan-out): Partition the input into disjoint chunks and process each independently— e.g., run security, performance, and style checks on a codebase simultaneously.

Voting (redundancy): Issue the same prompt N times with different seeds or temperatures, then select the best result via majority vote, reward-model scoring, or LLM-as-judge.



> **Use Cases:** Multi-aspect code reviews, guardrail checks.

Workflow Patterns (Cont.)

5. Orchestrator–Workers

A central LLM dynamically analyzes a task, breaks it down into subtasks, delegates to specialized worker agents, and synthesizes the collected worker outputs into a final result. [6]

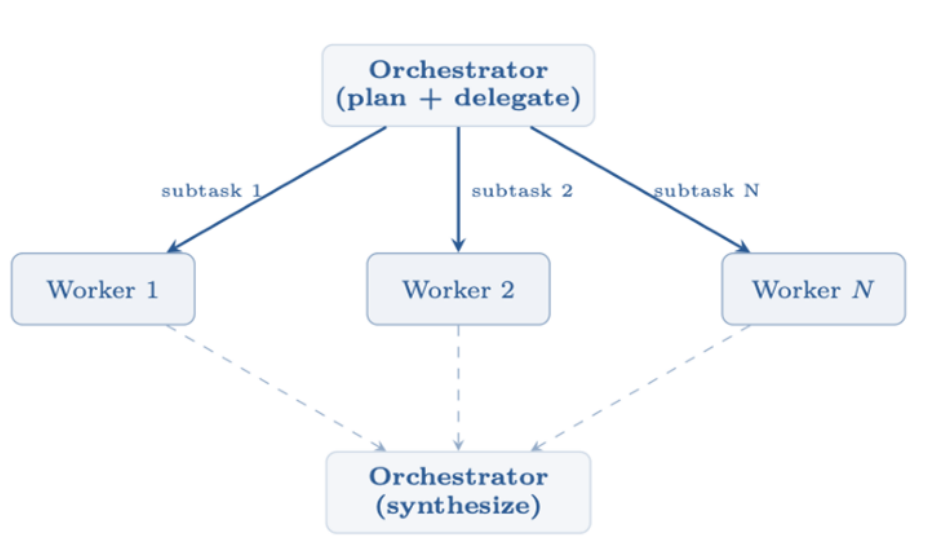


Fig: Orchestrator-workers: the LLM decides how to decompose the task and synthesizes worker results. [6]

> **Example Use Cases:** Complex software development, multi-domain compliance analysis.

6. Evaluator–Optimizer

Creates an iterative loop where a Generator LLM produces output and an Evaluator LLM assesses it against quality criteria, providing actionable feedback until thresholds are met.[6]

When to use: Tasks with clear quality criteria—code that must pass tests, translations that must preserve meaning, writing that must match a style guide.

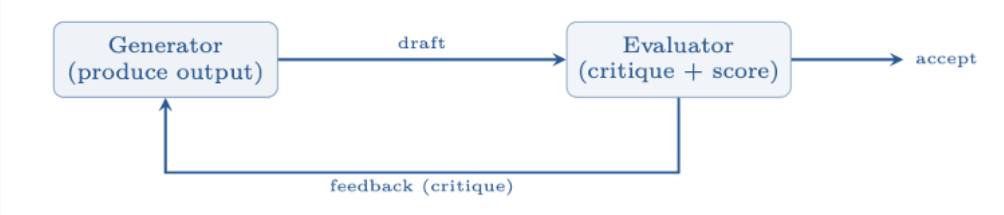


Fig: Evaluator-optimizer: iterative refinement without training.[6]

> **Example Use Cases:** Code generation with test pass verification, high-stakes translation,

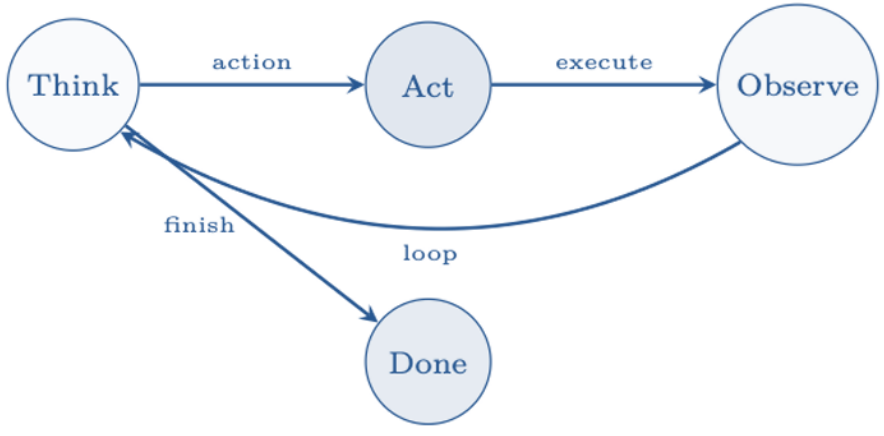
Autonomous Agent Patterns

Systems where the LLM-Based Agent independently determines execution paths, tool selection, reflection loops, and agent collaboration.

1. ReAct (Reason & Act)

It combines reasoning and action in a dynamic, iterative loop where a single agent observes the environment, formulates a plan, executes an action using available tools or APIs, and evaluates the resulting feedback to decide the next step. By constantly re-evaluating its context, the model can course-correct on the fly, retry failed tool calls, or adjust its strategy based on intermediate outputs. [6]

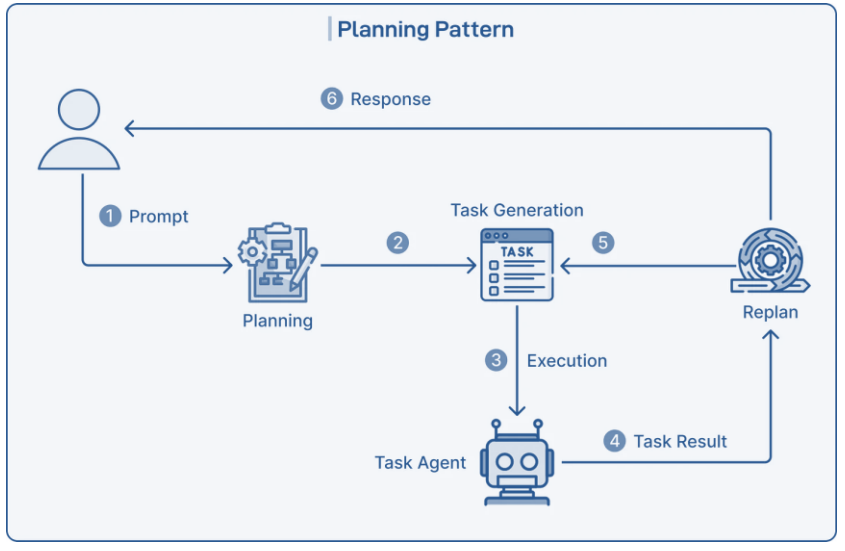
$$\text{Thought}_t \rightarrow \text{Action}_t \rightarrow \text{Observation}_t \rightarrow \text{Thought}_{t+1} \rightarrow \cdots$$



Example Use Cases: Q&A, Code generation & debugging

2. Planning Agent

It generates an explicit plan for achieving a goal before beginning execution. The plan contains tasks and dependencies, while the agent can revise the plan when new information, failures, or unexpected results are encountered during execution.[6]



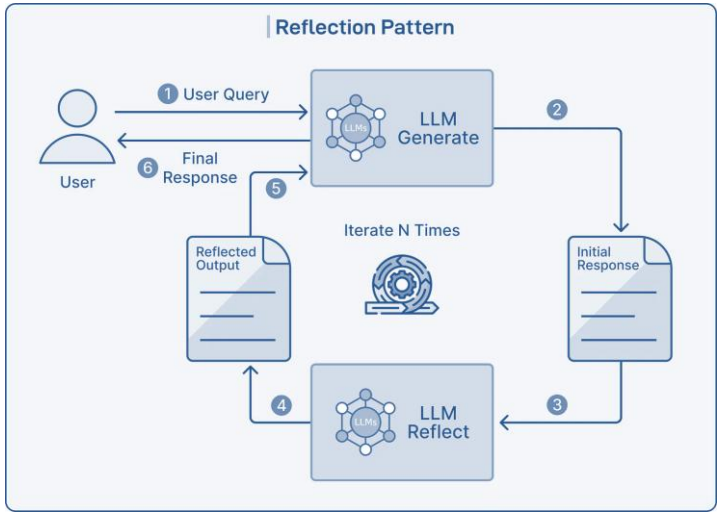
Example Use Cases: Content generation, document analysis, data transformation

Autonomous Agent Patterns (Cont.)

Systems where the LLM-Based Agent independently determines execution paths, tool selection, reflection loops, and agent collaboration.

3. Reflection & Self-Critique

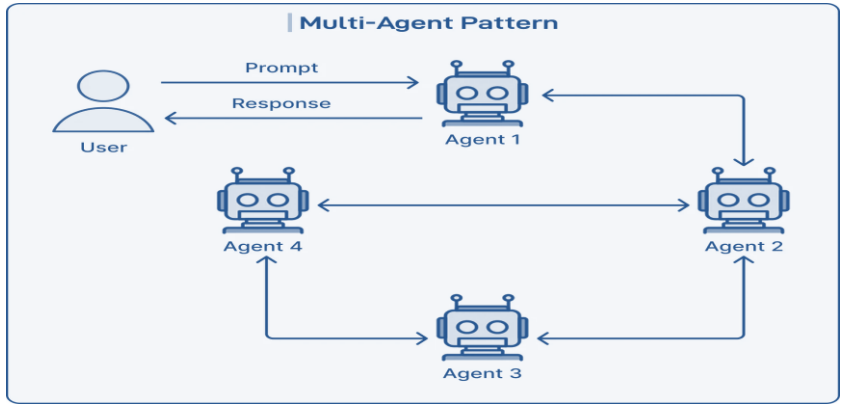
It introduces a self-evaluation stage where the agent examines its previous actions or output, identifies errors or inefficiencies, and modifies its strategy. The reflection can also be stored as memory and reused during subsequent attempts [6]



Example Use Cases: Customer support automation

4. Multi-Agent Swarm

A **Multi-Agent Swarm** relies on a manager agent to orchestrate a ecosystem of domain-expert sub-agents working together toward a high-level goal. The manager breaks down complex user objectives into specific tasks and assigns them to specialized roles—such as a Coder for implementation, a Researcher for context gathering, and a Reviewer for quality assurance—who can debate, review each other's work, and iteratively refine deliverables. [6]



Example Use Cases: Software development teams, Threat analysis & support

Harness Engineering

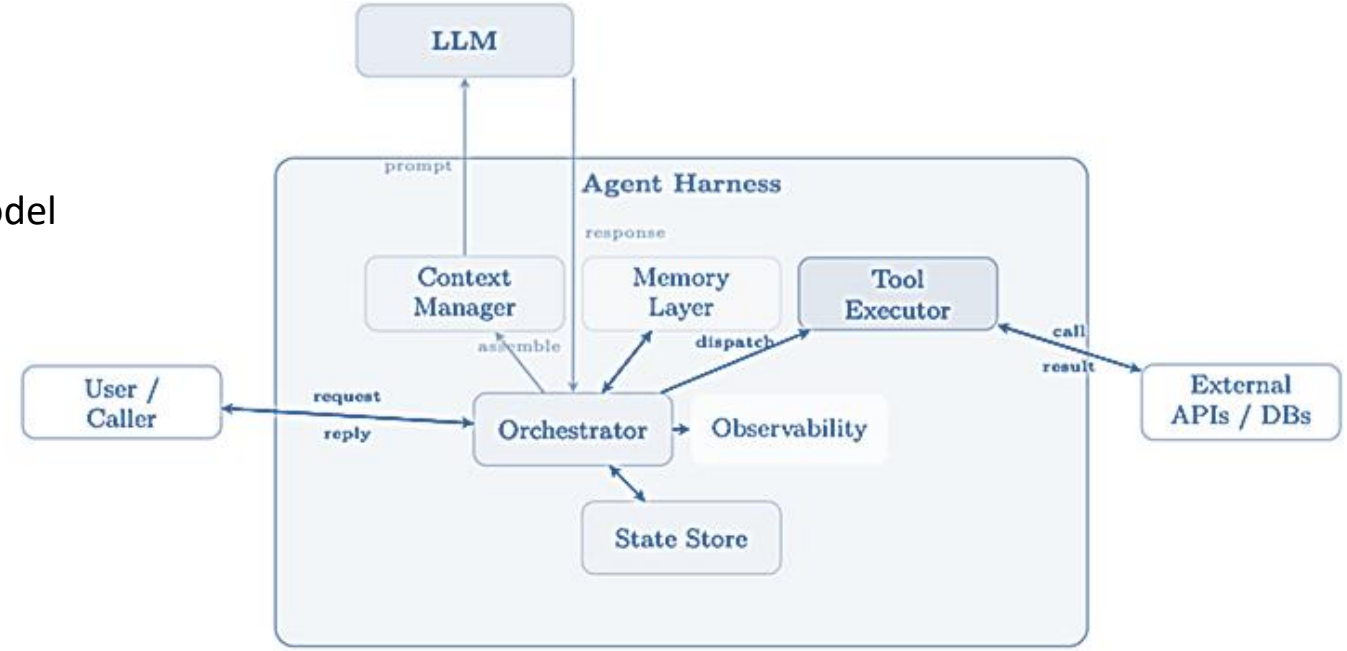
An LLM alone is not sufficient for reliable long-running software development. **Harness Engineering** focuses on designing the surrounding environment—context, tools, memory, state, constraints, verification, and feedback loops—so agents can effectively execute complex tasks.[14]

There are things we want an agent to do that a model cannot do out of the box. This is where a harness comes in. A **harness** is every piece of **code, configuration, and execution logic** that isn't the model itself. A raw LLM model is not an agent. But it becomes one when a harness gives it things like state, tool execution, feedback loops, and enforceable constraints. [19]

Concretely, a harness includes things like:

- System Prompts
- Tools, Skills, MCPs + and their descriptions
- Bundled Infrastructure (filesystem, sandbox, browser)
- Orchestration Logic (subagent spawning, handoffs, model routing)
- Hooks/Middleware for deterministic execution (compaction, continuation, lint checks)

Agent = Model + Harness



Harness Engineering

In simple terms, harness engineering is the practice of designing the full environment that an AI agent operates within: the rules it follows, the checks it must pass, and the feedback loops that prevent the same mistake from recurring.

The harness enforces a clean separation of concerns:

- **Reasoning:** Delegated entirely to the LLM; the harness does not second-guess model outputs.
- **Execution:** The harness dispatches tool calls, manages I/O, and enforces sandboxing.
- **Memory:** The harness maintains short-term (context window), working (scratchpad), and long-term (vector store / database) memory.
- **Communication:** The harness handles message routing between agents, users, and external services.
- **Observability:** The harness instruments every step for logging, tracing, and debugging [6]

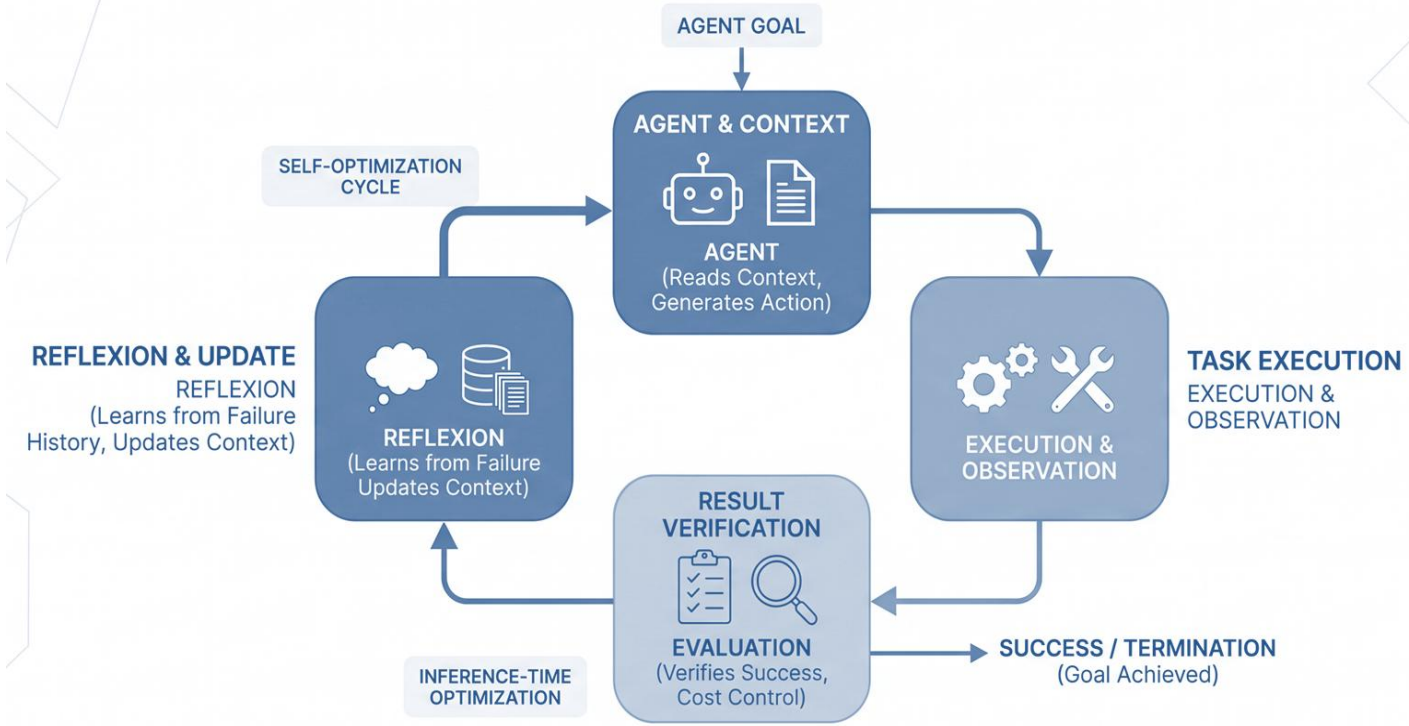
Why Separate Concerns?

A language model is a function f_θ : tokens → tokens. It has no persistent state, no ability to call APIs, and no awareness of time. The harness is the "operating system" that gives the model a body—persistent memory, actuators (tools), and a scheduler (orchestrator). Just as an OS abstracts hardware from applications, the harness abstracts infrastructure from the model [6]

Loop Engineering

The **Orchestration Harness** coordinates everything: it decides when to retrieve, when to call tools, when to delegate to sub-agents, and when to ask the human for guidance, whereas **Loop Engineering** formalizes the harness's retry-and-verify cycles as inference-time optimization, ensuring convergence and cost control.

It reflects a structural reality: When a single agent run may last an hour and modify dozens of files, the highest-leverage engineering is no longer in the prompt—it is the **Loop** that keeps the agent productive, verified, and on-goal throughout. [6]



From Prompt to Loop: The Engineering Progression[6]

1. Prompt engineering: Optimize what you say to the model (2022–24)

2. Context engineering: Optimize everything the model sees (2025)

3. Harness engineering: Optimize the environment the agent runs in (2025–26)

4. Loop engineering: Optimize the cycle that keeps the agent working toward a goal (2026)

Loop Engineering

A well-engineered agent loop is structurally identical to an RL optimization process operating at inference time—without gradient updates to the model weights:

$$
\begin{aligned}
s_t &= \text{context}(s_{t-1}, a_{t-1}, o_{t-1}) && \text{(state = accumulated context)} \\
a_t &= \pi_\theta(s_t) && \text{(action = model generation)} \\
o_t &= \text{env}(a_t) && \text{(observation = tool/environment feedback)} \\
r_t &= \text{verify}(s_t, a_t, o_t) && \text{(reward = verification signal)}
\end{aligned}
$$

The **loop** continues until $\mathbf{r_t}$ exceeds a success threshold or a termination condition triggers. The "policy" $\boldsymbol{\pi_\theta}$ is not updated via gradients; instead, the ***state*** is updated—observations, error messages, and reflections are appended to the context, conditioning the same frozen model to produce better actions on subsequent iterations. This is precisely the mechanism behind **Reflexion**, the model improves across iterations not by learning new weights but by reading its own failure history. [6]

Reflection in Practice (Reflexion)

Example: After three failed attempts to solve a coding problem, the agent reflects:

1. Retrieves the three failure episodes from episodic memory.
2. Generates an insight: "I keep forgetting to handle the edge case where the input list is empty."
3. Stores this insight in semantic memory.
4. On the next attempt, retrieves the insight and explicitly checks for empty inputs.

This is the core mechanism of **Reflexion**; learning via self-reflection. [6]

Harness Engineering provides the controlled environment around agents—giving them the right context, tools, permissions, testing, guardrails, and observability for reliable execution. **Loop Engineering** enables continuous autonomous operation by repeatedly discovering tasks, executing them, verifying results, learning from feedback, and re-planning when needed. Together, they can make our Autonomous SDLC more **reliable, adaptive, and self-correcting**.

MCP & Why Do We Need It?

- **The Core Problem:** Every time a new LLM agent framework appears, developers must re-implement connectors to the same tools: file systems, databases, web search, code execution, calendar APIs. This is wasteful, error-prone, and creates a maintenance burden that scales quadratically with the number of agents and tools.

  Also LLM knowledge is frozen at training time & LLMs Agents cannot inherently interact with the outside world (real-time data, business systems, APIs).

- **The Solution (MCP):** The Model Context Protocol is an open standard that enables developers to build secure, two-way connections between their data sources and AI-powered tools. The architecture is straightforward: developers can either expose their data through MCP servers or build AI applications (MCP clients) that connect to these servers [18]

Here is a simplified look at how MCP would handle this:

1. Discover Tools: AI realizes it needs external help and finds the database and email tools registered on MCP.

2. Fetch Data: AI requests the sales report via MCP; the database server fetches and returns the report.

3. Send Email: AI hands the report to the email tool, which sends it to the manager.

4. Confirm: AI tells you the task is finished. [18][6]

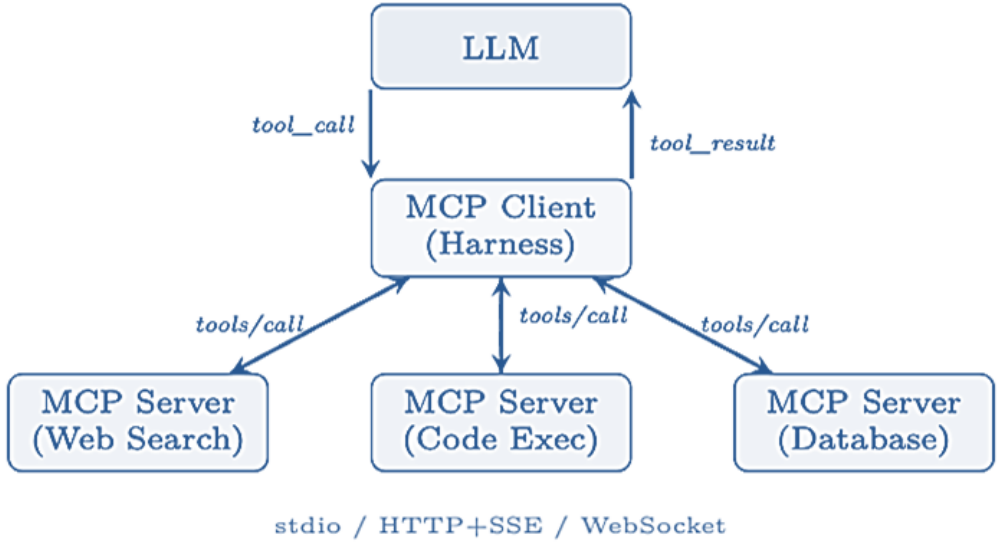


Fig: MCP architecture. The harness acts as an MCP client, routing tool calls to specialized MCP servers over standardized transports.[6]

How MCP works:

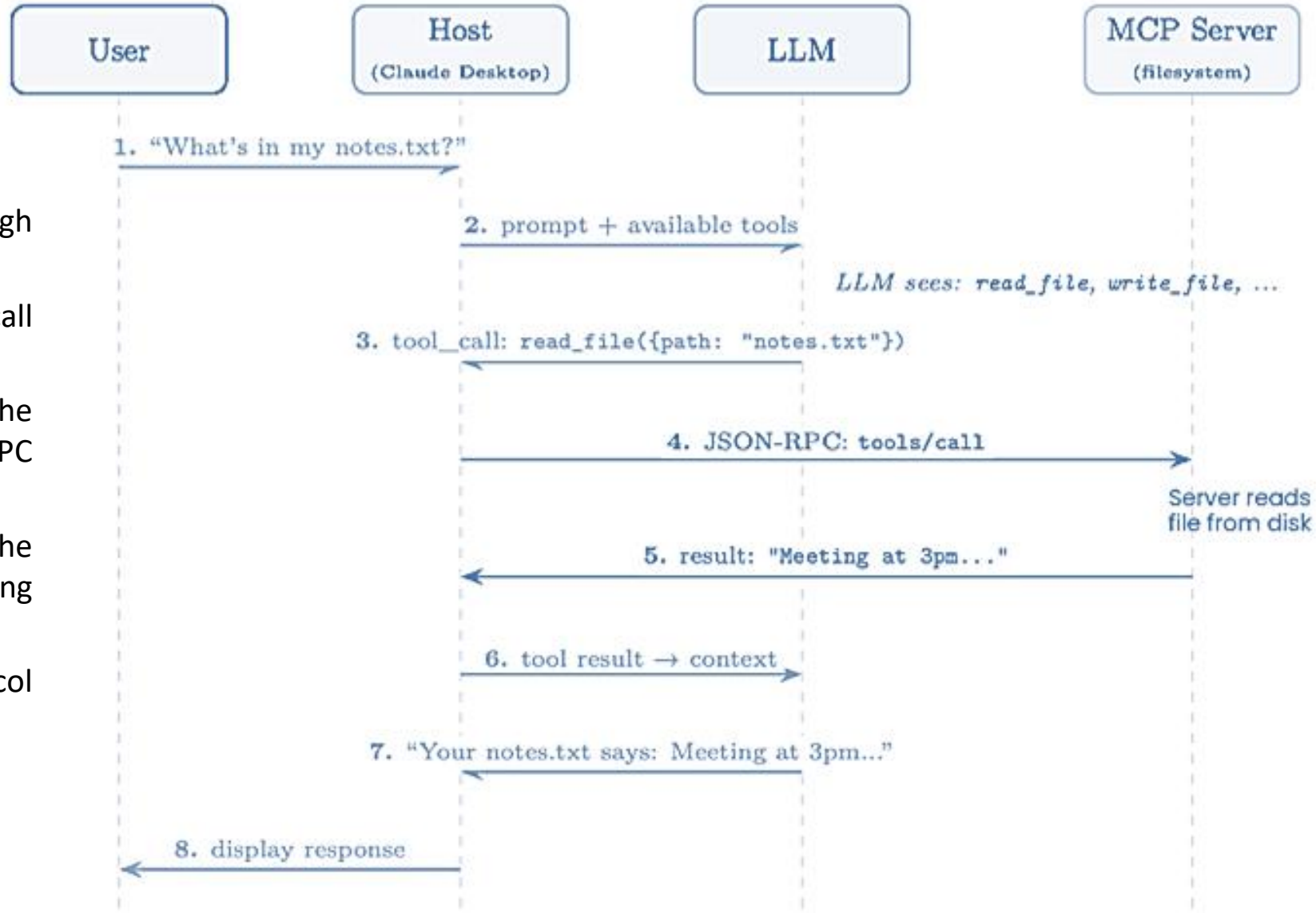


Fig: A single user request flows through the **Host**, **LLM**, and **MCP Server**.

- The LLM decides which tool to call (step 3);
- the Host routes the call to the appropriate server via JSON-RPC (step 4);
- the result flows back through the LLM for natural-language formatting (steps 5–7).
- The user never sees the protocol machinery. [6]

Agent-to-Agent Communication (A2A)

- **The Core Problem:** As large language models evolve from isolated assistants into collaborative networks of specialized agents, the question of how agents talk to each other becomes as important as how they reason internally.
- **The Solution (A2A):** In April 2025, Google (with contributions from over 50 technology partners) released the Agent to-Agent (A2A) Protocol [113], an open specification for interoperable communication between AI agents. [6]

A2A is built on five core principles:

- **Opaque Execution:** Agents interact through defined interfaces without needing to know how another agent works internally.
- **Enterprise Ready:** Supports authentication, security, auditing, and compliance.
- **Modality Agnostic:** Supports text, files, and structured data.
- **Built on Standards:** Uses common protocols such as HTTP, JSON-RPC, SSE, and gRPC.
- **Async-First:** Supports long-running tasks through polling and notifications without keeping connections open. [6]

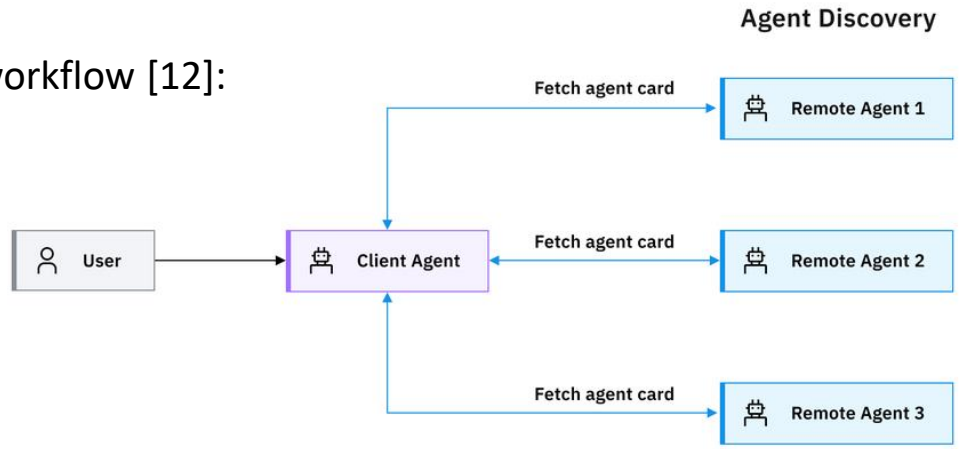


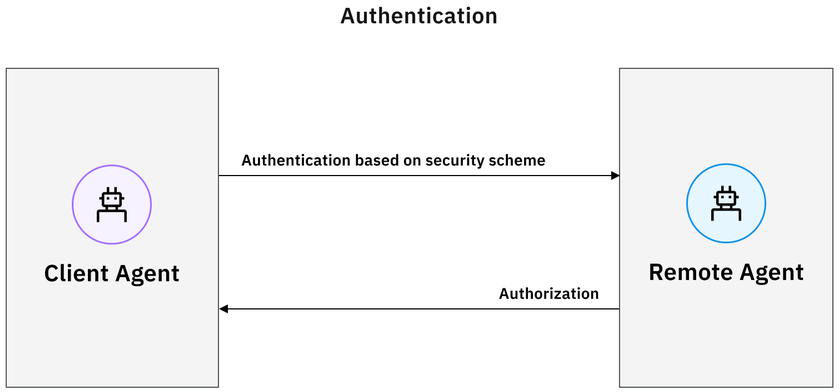
Agent-to-Agent Communication (A2A)

The A2A protocol follows a client-server model setup with a three-step workflow [12]:

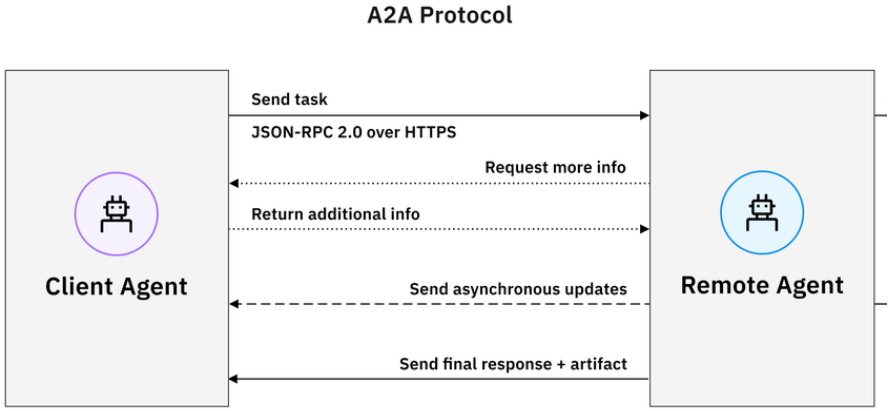
- Discovery
- Authentication
- Communication

1. The A2A workflow begins when an entity initiates a request to the client agent. A client agent then discovers/looks for remote agents and fetching their agent cards to determine the best fit for the task. [12]





2. Once the client agent identifies a remote agent capable of fulfilling the designated task, it then goes through authentication according to the security scheme (such as API keys, OAuth 2.0) indicated in the agent card.[12]



3. Communication starts with a client agent sending a task to the chosen remote agent. Agent-to-agent communication occurs over HTTPS for secure transport, with JSON-RPC (Remote Procedure Call) 2.0 as the format for data exchange. [12]

Agent Skills: Composable Capabilities

What Is a Skill?

A skill is a self-contained capability module that gives an agent expertise in a specific domain or task. Unlike a raw tool (which exposes a single function), a skill encompasses:

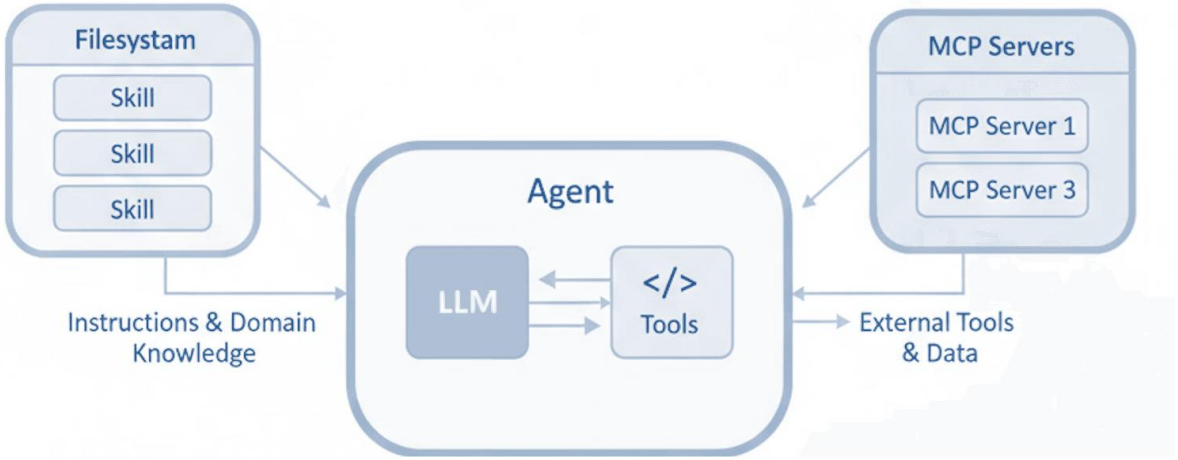
- **System prompt augmentation:** Domain-specific instructions, constraints, and persona elements injected into the agent's context.
- **Tool bindings:** One or more tools the skill requires (APIs, MCP servers, local commands).
- **Knowledge:** Reference material, examples, or few-shot demonstrations the agent needs to execute the skill correctly.
- **Workflow logic:** Multi-step procedures, decision trees, or conditional flows that guide the agent through complex tasks.
- **Guardrails:** Skill-specific safety constraints, output format requirements, and validation rules. [6]



Agent Skills (Cont.)

Here is a **pseudo-representation** of **a pre-defined AI Skill** declaration and its invocation by an Agent.

The Skill encapsulates domain-specific instructions, knowledge, tools, workflow logic, and guardrails into a reusable capability. The Agent can select and invoke the Skill when the corresponding task is required.




```
# Pre-defined skill
code_review = Skill(
    name="CodeReview",
    instructions="Review code for correctness, security and quality.",
    knowledge=["coding_standards", "security_guidelines"],
    tools=["static_analyzer", "test_runner"],
    workflow=["analyze", "test", "report"],
    guardrails=["verified_findings", "structured_output"]
)

# Agent
developer_agent = Agent(
    name="Developer",
    model="LLM",
    skills=[code_review],
    memory="project_memory"
)

# Agent uses the skill
result = developer_agent.run(
    skill="CodeReview",
    input=repository
)
```

Evaluating the Ai Agents

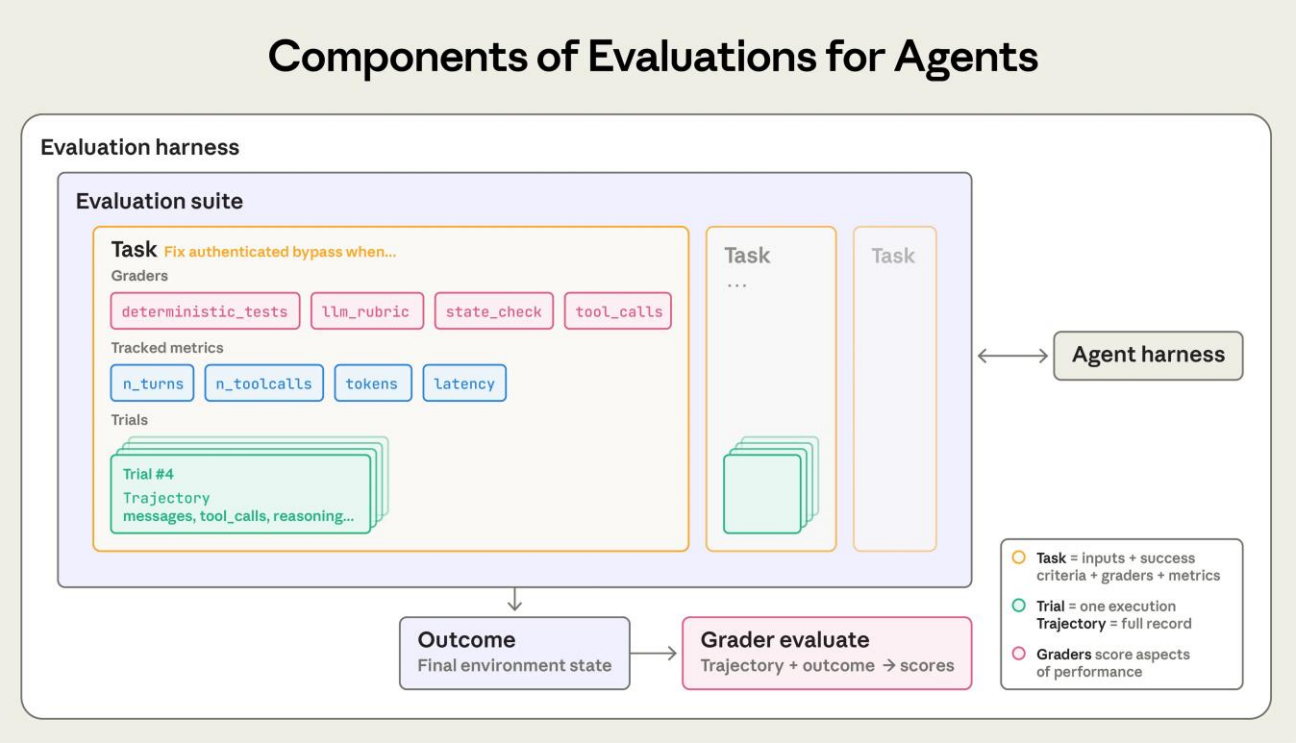
The important finding from current agent-evaluation literature is that **agent evaluation must assess more than the final answer**. Agents act over multiple steps, use tools, modify state, and can fail in different places. Anthropic explicitly recommends evaluating the **task, trajectory/transcript, and final environment outcome**, with multiple trials because agent behavior is variable.

For agents, the evaluation should examine:

- **What the agent produced**
- **How it acted**
- **Whether the task actually succeeded**
- **How efficiently and safely it behaved.** [20]

DeepEval is an open-source evaluation framework used by developers to unit-test, trace, and score Large Language Model (LLM) applications and AI agents. For AI agents, it goes beyond simple input-output checks to evaluate complex multi-step reasoning, tool selection, and overall task execution trajectories [21]

DeepEval simplifies building an Evaluation Harness, an **evaluation harness** is the infrastructure that runs evals end-to-end. It provides instructions and tools, runs tasks concurrently, records all the steps, grades outputs, and aggregates results.



Evaluating the Ai Agents (Cont.)

An **evaluation suite** is a collection of tasks designed to measure specific capabilities or behaviors. Tasks in a suite typically share a broad goal. For instance, a customer support eval suite might test refunds, cancellations, and escalations.[20]

It supports 10+ popular agent frameworks : LangChain, LangGraph, OpenAI Agents, Pydantic AI, CrewAI, Google ADK, and more.[21]

When building agent evaluations, we use the following definitions:

- A **task** (a.k.a **problem** or **test case**) is a single test with defined inputs and success criteria.
- Each attempt at a task is a **trial**. Because model outputs vary between runs, we run multiple trials to produce more consistent results.
- A **grader** is logic that scores some aspect of the agent's performance. A task can have multiple graders, each containing multiple assertions (sometimes called **checks**).
- A **transcript** (also called a **trace** or **trajectory**) is the complete record of a trial, including outputs, tool calls, reasoning, intermediate results, and any other interactions.
- The **outcome** is the final state in the environment at the end of the trial. [20]

AI agents consist of interconnected layers that each require distinct evaluation approaches: [21]

Layer	What It Does	Evaluation Scope	Key Metrics
Reasoning Layer	Plans tasks, creates strategies, decides what to do	Trajectory	PlanQualityMetric, PlanAdherenceMetric
Action Layer	Selects tools, generates arguments, executes calls	Component-level	ToolCorrectnessMetric, ArgumentCorrectnessMetric
Execution Layer	Orchestrates the full loop, completes objectives	Trajectory	TaskCompletionMetric, StepEfficiencyMetric

The Traditional SDLC

The Traditional Software Development Life Cycle (SDLC) is the structured framework that engineering teams follow to plan, design, build, test, and deploy software applications from initial concept.

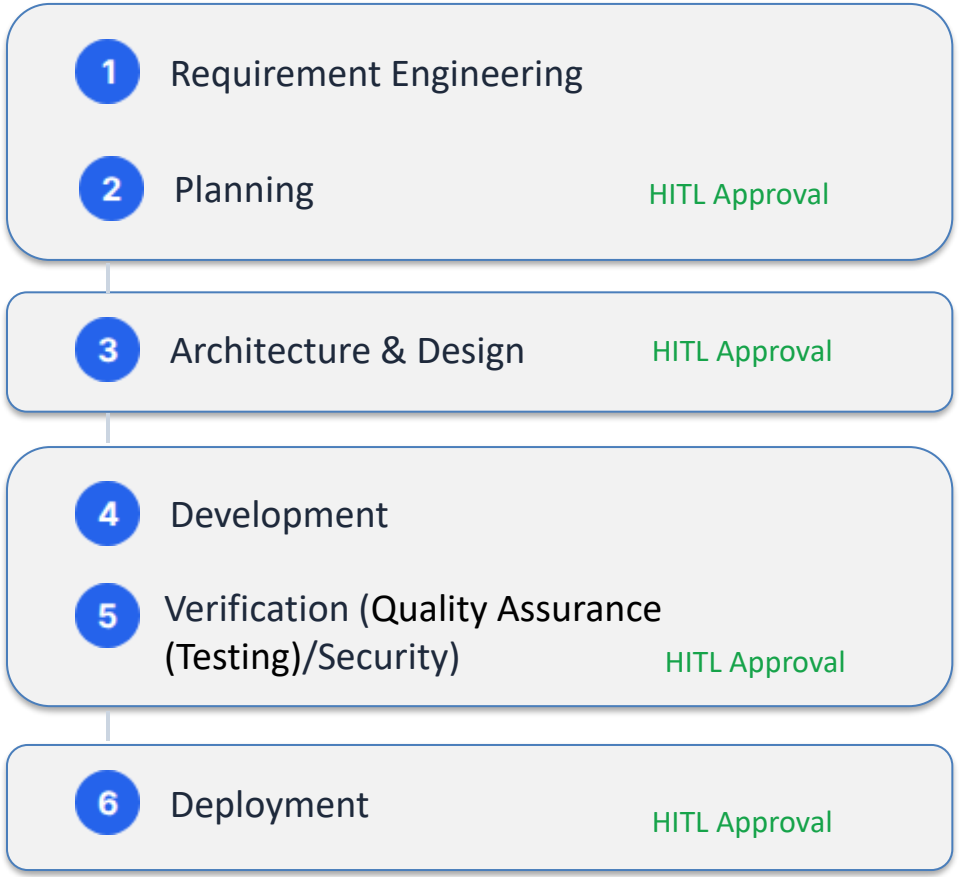


This Manual SDLC execution creates severe friction—engineers waste most of their time on administrative handoffs, environment setups, regression testing, and ticketing overhead instead of writing core logic.[5]

So, Our Aim is to automate the Traditional SDLC with Agentic AI, which will replaces these static bottlenecks with continuous, self-healing pipelines that autonomously execute tests, patch build failures, flag vulnerabilities, and manage releases. This **eliminates human error, compresses cycle times from months to hours, and frees development teams to focus on high-value architecture and innovation.**

Proposed Architecture: Modular Pipeline

A modular pipeline with integrated **Human-in-the-Loop (HITL)** governance gates to ensure enterprise safety.

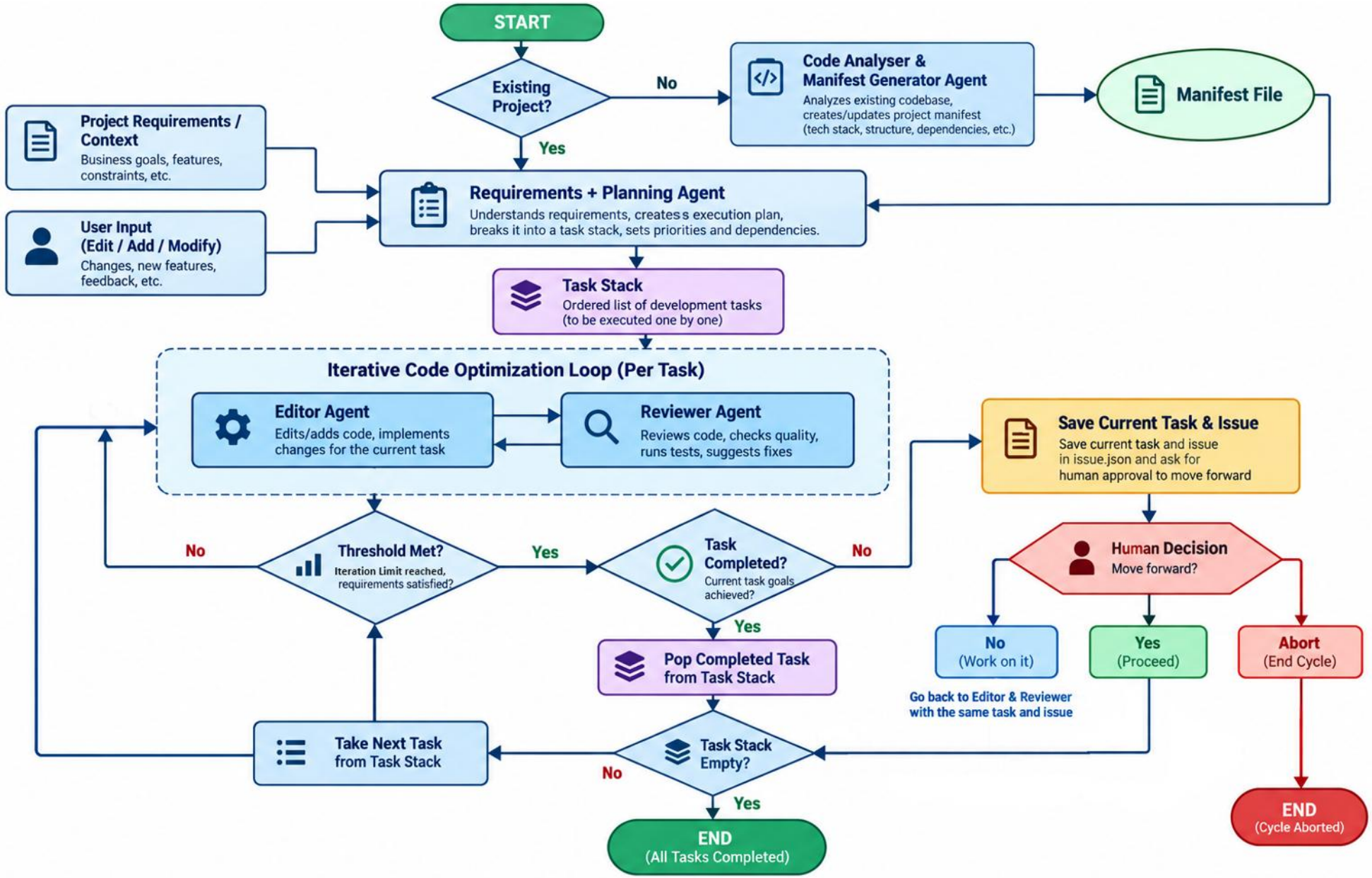


Continuous Loop

7. Monitoring & Learning: Consumes production logs, metrics, and user feedback to detect errors and optimize prompts.

8. Organizational Memory: Captures all requirements, code, reviews, and bug fixes into a Knowledge Graph for cross-sprint learning.

Proposed Architecture: Modular Pipeline



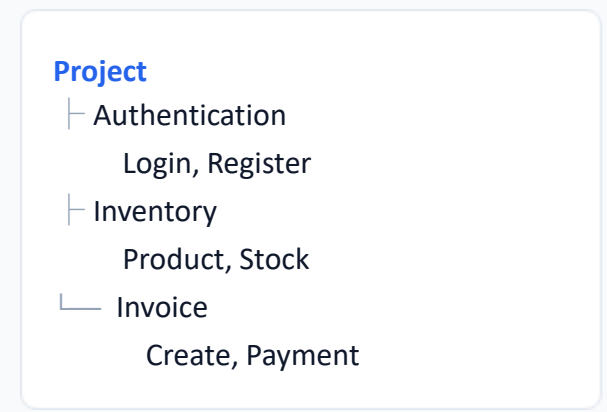
Core Innovation: Hierarchical Software Knowledge Graph (HSKG)

Beyond Traditional Memory:
A Context-Aware Project Memory

The Problem: Current AI agents retrieve entire repositories or large documents, resulting in high token consumption and irrelevant context. [1]

Our Solution: Our proposed HSKG stores the software project as a hierarchical graph, enabling agents to retrieve only the smallest relevant subgraph required for a task.

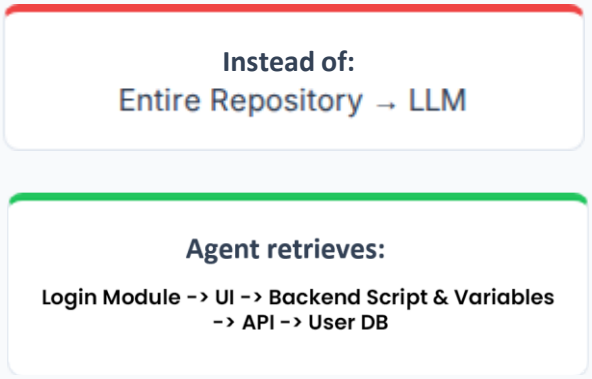
Project Graph



Purpose

- Represents the complete SDLC structure
- Maintains hierarchy between project components
- Enables modular navigation

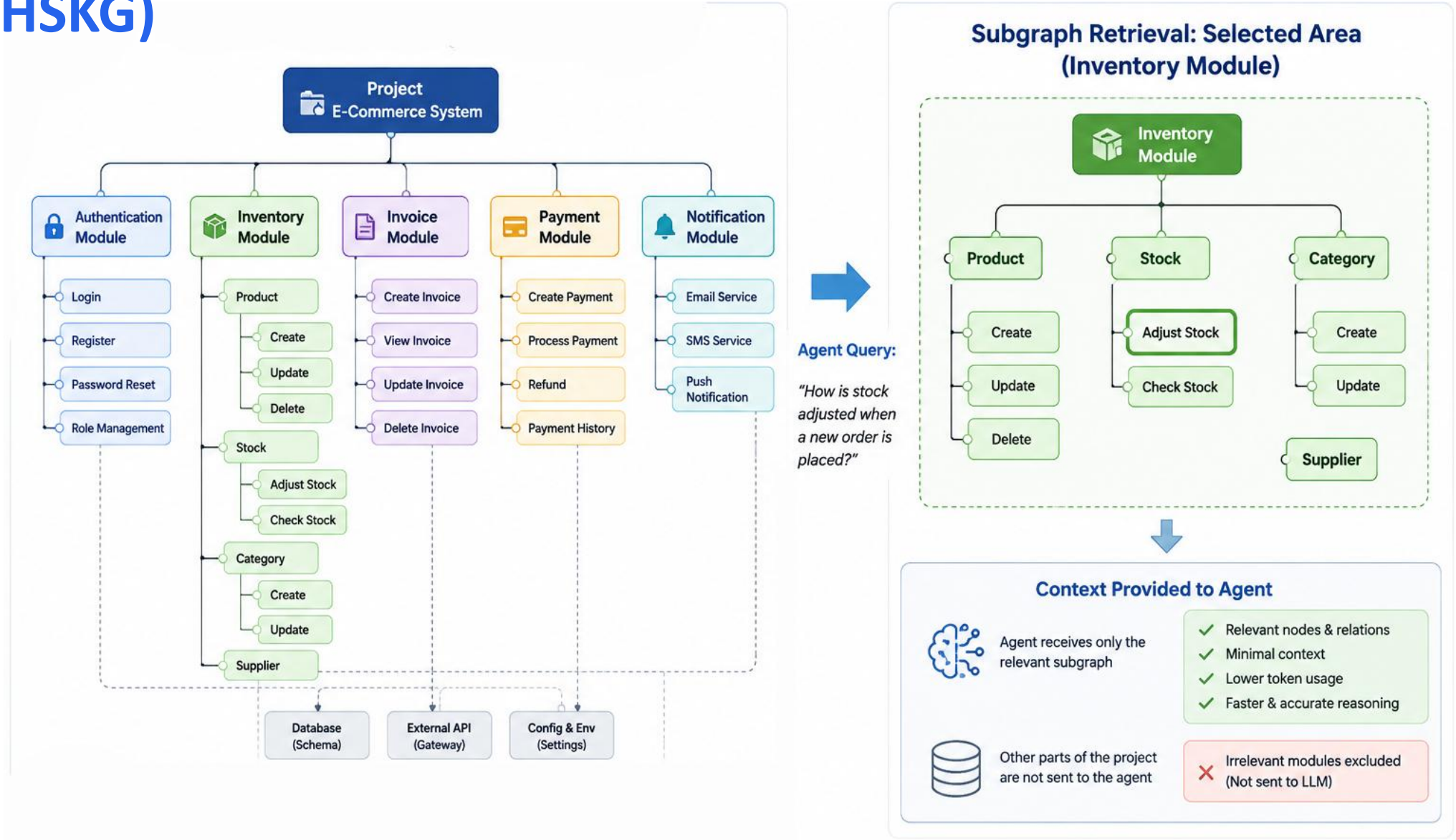
Context-Aware



Benefits

- Minimal context
- Lower token usage
- Faster reasoning
- Better accuracy
- Reduced hallucination

Core Innovation: Hierarchical Software Knowledge Graph (HSKG)



Existing SDLC Automations

While existing autonomous systems are highly capable at code execution, they lack **long-term organizational memory** and struggle with end-to-end orchestration.

System	Requirements	Architecture	Coding & Test	Deployment	Org. Memory
Devin AI [15]	**Partial,** Prompt-driven only Partial	**Partial**, Ad-hoc planning Partial	**Yes,** Autonomous execution Yes	**Partial,** Sandboxed environments Partial	**No,** Session-limited context No
MetaGPT [11]	**Yes,** Multi-agent parsing Yes	**Yes,** Structured PRD generation Yes	**Partial,** Lacks dynamic env. feedback Partial	**No**, Manual intervention req. No	**Partial,** Static file-based memory Partial
GitHub Copilot Workspace[13]	**Partial**, Spec/Issue refinement Partial	**Partial**, File-level plan generation Partial	**Yes**, Autonomous code generation & test runs Yes	**No**, Hands off at PR stage No	**Partial**, Repository & PR history context Partial
Proposed System	**Yes,** Interactive elicitation Yes	**Yes,** Context-aware design Yes	**Yes,** Agentic orchestration Yes	**Yes,** CI/CD pipeline integrated Yes	**Yes,** Long-term vector, HKG, retrieval Yes

Source: Compiled from official documentation and validated using publicly available implementation benchmarks.

Agentic Frameworks

Framework	Execution Model	Core Strengths	Key Limitations	Ideal Use Cases	Local / Offline
LangChain [7]	Linear chains & tool pipelines	Large integration ecosystem; RAG & tool abstraction; rapid development	High abstraction; limited control over complex loops; state management can become cumbersome	RAG, Q&A, tool-based LLM apps	⭐⭐⭐⭐⭐ Excellent — Ollama, llama.cpp, Chroma, Qdrant
LangGraph [8]	Stateful graph with cycles & conditional flows	Fine-grained state control; checkpointing; fault tolerance; complex loops	Steeper learning curve; more setup; explicit state management	Complex agents, multi-agent systems, SDLC automation, HITL	⭐⭐⭐⭐⭐ Excellent — self-hosted models & local stores
CrewAI [9]	Role-based sequential / hierarchical teams	Simple agent-role abstraction; delegation; fast multi-agent prototyping	Less flexible for highly dynamic workflows; coordination overhead	Multi-agent research, content teams, structured workflows	⭐⭐⭐⭐ Strong — Ollama, LiteLLM, vLLM
OpenAI Agents SDK [10]	Handoffs, tools & guardrail-driven execution	Simple abstractions; guardrails; tracing; agent handoffs; sandbox support	Limited built-in long-term memory/graph persistence; ecosystem-oriented conventions	Multi-agent services, real-time/voice agents, sandboxed coding	⭐⭐⭐⭐ Very Good — local models through compatible endpoints

Source: *Compiled from official model documentation and validated using publicly available implementation benchmarks and deployment experiences.*

Local Offline Execution Strategy

Deploying via offline models ensures data privacy. Model selection is based on VRAM footprint, Training data freshness, Suitable for Agentic Operations.

Models	Approx. VRAM	Context Window	Best Use Case
Mistral Small 3.2 24B	~15–17 GB	**128K**	General agents, planning, tool use
Qwen3 30B-A3B	~18–20 GB	**32K**	**Orchestrator, planning, reasoning**
Qwen2.5-Coder 32B	~18–20 GB	**128K**	**Developer, coding & debugging**
Qwen3 32B	~19–22 GB	**32K**	Architect, reasoning, complex agents
Gemma 4 31B	~19–21 GB	**256K**	Multimodal, coding, reasoning & agents

Source: *Compiled from official model documentation and validated using publicly available implementation benchmarks and deployment experiences.*

Conclusion

We propose an **Autonomous Software Development Lifecycle (A-SDLC)** framework that transforms traditional software development into an **AI-orchestrated, adaptive engineering process**.

Instead of relying on isolated coding assistants, the framework coordinates **specialized AI agents, reusable skills, persistent project memory (HSKG), tools, and feedback loops** across the complete SDLC—from **requirements and architecture to development, testing, deployment, and continuous improvement**.

Expected Outcomes

- **End-to-end SDLC automation** through coordinated specialized agents
- **Context-efficient development** through HSKG-based selective retrieval
- **Reliable and adaptive execution** through harnesses, verification, and feedback loops
- **Improved traceability** of requirements, decisions, code, tests, and changes
- **Reusable engineering knowledge** through persistent project memory and skills
- **Reduced repetitive human effort** while retaining human oversight for critical decisions

The goal is not to replace software engineers, but to build an AI-orchestrated engineering system that can continuously plan, execute, verify, and improve software development.

References

Research Papers & Book

[1] He, J., Treude, C., & Lo, D. (2025). *LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision...* ACM TOSEM.

[2] Apostolou, S. A., et al. (2026). *Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the SDLC*. arXiv.

[3] *Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence...* (2026). arXiv.

[4] *Veselka Sasheva Petrova-Dimitrova, "Classifications of intelligence agents and their applications," Fundamental sciences and applications, Vol. 28, No. 1, 2022.*

[5] Sommerville, I. *Software Engineering (10th Edition)*.

[6] The Hitchhiker's Guide to Agentic AI: From Foundations to Systems by Haggai Roitman

Articles & Official Documentations

[7] LangChain Documentation.

[8] LangGraph Documentation.

[9] CrewAI Documentation.

[10] OpenAI Agents SDK Documentation.

[11] Hong, S., et al. (2024). MetaGPT: Meta Programming for Multi-Agent Collaborative Framework.

[12] IBM: https://www.ibm.com/think/topics/agent2agent-protocol

[13] GitHub. *"Copilot Workspace: AI-Native Developer Environment."*: https://githubnext.com/projects/copilot-workspace/

[14] OpenAi - https://openai.com/index/harness-engineering/

[15] Cognition AI – Devin: AI Software Engineer.

[16] Microsoft Copilot: Microsoft-copilot/for-individuals/do-more-with-ai/general-ai/understanding-ai-agents-vs-chatbots?

[17] IBM: https://www.ibm.com/think/topics/ai-agents

[18] Google Cloud**:** cloud.google.com/discover/what-is-model-context-protocol

[19] Langchain: https://www.langchain.com/blog/the-anatomy-of-an-agent-harness

[20] Anthropic: https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

[21] DeepEval: https://deepeval.com/guides/guides-ai-agent-evaluation

Thank You